

Contenido por Estación

Documentación técnica de la solución

Panel Frontend React / CMS — listado de estaciones, servicios asociados y gestión de estado operativo

Proyecto	AppTerpel
Área	Dirección Canales Digitales
Flujo	Prueba Técnica — Ingeniero de Aplicaciones de Negocio
Documento	Documentación técnica de la solución
Versión	1.0
Fecha	Agosto 2026

Tabla de contenido

1. Introducción.....	4
2. Objetivo	4
2.1 Objetivo general.....	4
2.2 Objetivos específicos	4
3. Alcance del proyecto.....	4
4. Arquitectura de la solución	5
4.1 Estructura por capas	5
4.2 Modelo de datos.....	6
5. Stack tecnológico	6
6. Funcionalidades implementadas	7
6.1 Requisitos base del enunciado.....	7
6.2 Funcionalidades adicionales	7
Experiencia de usuario.....	8
Confiabilidad	8
Productividad y mantenibilidad	8
7. Análisis y solución de bugs	8
7.1 Descripción del defecto	8
7.2 Problemas identificados.....	8
7.3 Solución implementada	9
8. Caso situacional: trabajo en equipo.....	9
8.1 ¿Cómo minimizar el impacto del cambio?.....	10
8.2 ¿Cómo coordinar el contrato con backend / CMS?	10
8.3 ¿Qué se haría para que estos cambios no rompan producción?	10
9. Estrategia de calidad	11
9.1 Testing	11
9.2 Accesibilidad	11
9.3 CI/CD y automatización.....	11
10. Manual de instalación	13
11. Manual de usuario	13
12. Conclusiones	14
Anexos	15
Anexo A. Documentación complementaria en el repositorio	15
Anexo B. Scripts disponibles	15
Anexo C. Datos de contacto del proyecto.....	15

1. Introducción

Este documento consolida la documentación técnica de la solución desarrollada para la Prueba Técnica de Ingreso orientada a Frontend, correspondiente al flujo “Prueba Técnica — Ingeniero de Aplicaciones de Negocio” de la Dirección de Canales Digitales, en el marco del proyecto AppTerpel.

La prueba solicitó construir un panel de “Contenido por Estación” que permitiera listar estaciones de servicio, consultar los servicios que ofrece cada una y modificar su estado operativo, evaluando tanto el dominio de fundamentos técnicos como la capacidad de análisis, resolución de problemas y comunicación dentro de un equipo de trabajo. Este documento describe la solución entregada, las decisiones de arquitectura tomadas, la estrategia de calidad aplicada y las instrucciones necesarias para instalar, operar y mantener la aplicación.

2. Objetivo

2.1 Objetivo general

Desarrollar una aplicación frontend en React que permita gestionar el contenido y el estado operativo de las estaciones de servicio, consultando de forma clara los servicios asociados a cada una, bajo una arquitectura de calidad profesional, con cobertura de pruebas automatizadas y prácticas de desarrollo colaborativo.

2.2 Objetivos específicos

- Listar las estaciones disponibles identificándolas por su stationId y permitir su selección para consultar el detalle de servicios asociados.
- Representar los servicios de cada estación mediante íconos, sin exponer el nombre del servicio como texto plano, manteniendo la información accesible para tecnologías de asistencia.
- Permitir modificar el estado operativo de una estación (activa / inactiva) desde la misma vista de listado.
- Aplicar una arquitectura por capas que aisle el acceso a datos, la lógica de negocio y la presentación, facilitando el mantenimiento y la evolución del proyecto.
- Incorporar una estrategia de pruebas (unitarias, de integración y de accesibilidad) que respalde la confiabilidad de la solución.
- Documentar el análisis y la corrección de un defecto de rendimiento relacionado con el ciclo de vida de componentes en React.
- Responder un caso situacional de coordinación entre frontend, diseño y backend/CMS ante un cambio de contrato y de interfaz.
- Automatizar la validación de calidad del código mediante integración continua (CI/CD).

3. Alcance del proyecto

La solución cubre la totalidad de los requisitos funcionales y no funcionales establecidos en el enunciado de la prueba técnica, y adicionalmente incorpora un conjunto de funcionalidades complementarias orientadas a acercar el entregable al estándar de un producto en producción (gestión de errores, accesibilidad automatizada, automatización de calidad y buenas prácticas de mantenimiento del repositorio). No fue necesario integrar un backend real: el enunciado entregó datos de ejemplo, que se normalizaron y se sirven a través de una capa de servicios que simula un API REST, diseñada para ser reemplazada por un backend real sin impactar el resto de la aplicación.

Requisito del enunciado	Estado
Listado de estaciones por stationId (useState + map)	Cumplido
Selección de estación y consulta de servicios asociados	Cumplido
Servicios representados por ícono, no por texto	Cumplido
Modificar el estado activa / inactiva de una estación	Cumplido
Fetching de datos con React Query	Cumplido
Arquitectura por capas (pages / components / services / hooks / types)	Cumplido
Accesibilidad mínima (labels, botones, foco)	Cumplido
Cache y manejo de stale data	Cumplido (plus)
Caso situacional de trabajo en equipo	Cumplido
Corrección del bug de useEffect / dependencias	Cumplido
Manual de instalación y de usuario	Cumplido
Presentación de la solución	Cumplido

4. Arquitectura de la solución

La aplicación sigue una arquitectura por capas con una regla de dependencia estricta: la capa de presentación nunca invoca directamente el acceso a datos; siempre lo hace a través de un hook. Esto permite que un cambio en el origen de los datos (por ejemplo, migrar de datos mock a un backend real) no impacte a los componentes visuales.

4.1 Estructura por capas

```
src/  
├── pages/           Composición de pantallas (StationsPage)  
├── components/      UI presentacional, organizada por feature  
│   └── Header/      Marca, logo y toggle de tema
```

SummaryCards/	Dashboard de métricas
StationSearch/	Buscador / filtro
StationSort/	Ordenamiento
ConfirmDialog/	Modal de confirmación
ErrorBoundary/	Recuperación ante errores
Toast/	Notificaciones
Theme/	Modo claro / oscuro
Skeletons/	Loading states
ExportCsvButton/	Exportación a CSV
StationList/	
ServicesPanel/	
StatusToggle/	
hooks/	React Query – useStations, useStationServices, useUpdateStationStatus
services/	Acceso a datos – api.ts, mockData.ts, queryClient.ts, queryKeys.ts, csvExport.ts
types/	Contrato de dominio (Station, ServiceKey, ContentItem, etc.)
test/	Setup y utilidades compartidas de testing

Regla de dependencia: pages → components + hooks → services → types. Ningún componente llama a fetch o a api.ts de forma directa; siempre pasa por un hook.

4.2 Modelo de datos

El enunciado entregó los datos de ejemplo con inconsistencias de sintaxis y de nomenclatura entre registros (stationId vs. idEstacion, idServicio vs. idServicios). Estos se normalizaron una única vez en la capa de servicios, de modo que el resto de la aplicación trabaje siempre contra un contrato de datos limpio y tipado.

Entidad	Campos
Station	id · name · stationId · status (active inactive)
StationServiceLink	id · stationId · serviceId
ServiceCatalogItem	id (s1..s4) · name
ContentItem (CMS)	id · name · stationId · status (draft published) · updatedAt

La entidad ContentItem corresponde al modelo editorial del CMS descrito en el enunciado; se modeló en la capa de tipos como contrato de referencia para el caso situacional (sección 8), dado que representa el contenido que el CMS expondrá cuando el layout evolucione de tarjetas a tabla.

5. Stack tecnológico

Categoría	Tecnología	Propósito
UI	React 18 + TypeScript	Componentes tipados, modo estricto

Categoría	Tecnología	Propósito
Build tool	Vite 5	Servidor de desarrollo y build de producción
Data fetching	TanStack Query (React Query) v5	Cache, cancelación, reintentos, mutaciones optimistas
Íconos	lucide-react	Íconos accesibles para los servicios
Testing	Vitest + React Testing Library + jsdom	Tests de componente, hook e integración
Accesibilidad	jest-axe	Tests automatizados de reglas WCAG
Calidad de código	ESLint + typescript-eslint	Consistencia y detección temprana de errores
Automatización local	Husky + lint-staged	Pre-commit hooks
Integración continua	GitHub Actions	Lint, tests con cobertura y build en cada push

No se utilizó un backend real. La capa `services/api.ts` simula un API REST: aplica latencia de red realista, respeta cancelación mediante `AbortSignal` y persiste los cambios en memoria durante la sesión, a partir de los datos de ejemplo del enunciado ya normalizados.

6. Funcionalidades implementadas

6.1 Requisitos base del enunciado

- Listado de estaciones por `stationId`, renderizado con `map` sobre estado gestionado con `useState` / `React Query`.
- Selección de una estación para consultar, en un panel independiente, los servicios que ofrece.
- Servicios representados mediante ícono (Baño, Cajeros, Soat, Tienda), con el nombre disponible como `aria-label` y `title` para lectores de pantalla.
- Modificación del estado operativo (activa / inactiva) de cada estación directamente desde el listado.
- Fetching con `React Query`, incluyendo cache y manejo de datos obsoletos (`stale data`) mediante `staleTime` y `gcTime` configurados por tipo de consulta.

6.2 Funcionalidades adicionales

Más allá del alcance mínimo solicitado, se incorporaron funcionalidades orientadas a la experiencia de usuario, la confiabilidad operativa y la productividad de quien use la herramienta en el día a día:

Experiencia de usuario

- **Dashboard de métricas** — tarjetas con el total de estaciones, activas e inactivas, recalculadas en tiempo real.
- **Búsqueda y filtro** — por nombre o código de estación, con atajo de teclado “/” para enfocar el buscador desde cualquier parte de la página.
- **Ordenamiento** — por nombre (A-Z / Z-A) o por estado.
- **Modo oscuro** — persistente en localStorage, con detección inicial de la preferencia del sistema operativo.

Confiabilidad

- **Confirmación antes de inactivar** — modal accesible (role="alertdialog", foco atrapado, cierre con Esc) para la acción de mayor riesgo operativo; reactivar, al ser de bajo riesgo, no la requiere.
- **Error Boundary** — de nivel de aplicación; ante un error inesperado de renderizado se muestra una pantalla de recuperación en vez de una página en blanco.
- **Manejo de deep-link inválido** — si la URL referencia una estación inexistente, se informa claramente en vez de dejar la interfaz en un estado inconsistente.
- **Notificaciones toast** — de éxito o error al cambiar el estado de una estación.

Productividad y mantenibilidad

- **Exportar a CSV** — descarga del listado de estaciones tal como se está visualizando, respetando el filtro y el orden activos.
- **Skeleton loaders** — en el listado de estaciones y el panel de servicios mientras cargan los datos.
- **Deep-linking por URL** — la estación seleccionada queda reflejada en ?station=<id>, permitiendo compartir el enlace directo o refrescar sin perder la selección.
- **Espacio de marca** — logo en el header con fallback automático a un ícono genérico si el archivo no está disponible.

7. Análisis y solución de bugs

7.1 Descripción del defecto

El enunciado propuso el siguiente fragmento de código con un defecto de rendimiento y de ciclo de vida:

```
useEffect(() => {  
  fetchContent(stationId);  
}, []);
```

7.2 Problemas identificados

1. Dependencias incorrectas: el arreglo de dependencias vacío hace que el efecto se ejecute una única vez, en el montaje. Si stationId cambia posteriormente, el efecto no vuelve a ejecutarse y la interfaz queda mostrando datos de la estación incorrecta (stale closure).

2. Fetch duplicado: sin cancelación, un remontaje en modo estricto de React o un cambio rápido de selección puede disparar más de una petición concurrente para el mismo dato.
3. Condiciones de carrera: si el usuario selecciona la estación A y luego, antes de que responda, selecciona la B, no hay garantía de que la respuesta de A no llegue después que la de B, dejando en pantalla datos de una estación distinta a la seleccionada.

7.3 Solución implementada

Se migró la lógica al hook `useStationServices`, apoyado en TanStack Query, incluyendo `stationId` como parte de la `queryKey`:

```
export function useStationServices(stationId: string | null) {
  return useQuery({
    queryKey: queryKeys.stationServices.byStation(stationId ?? ''),
    queryFn: ({ signal }) =>
      fetchStationServices(stationId as string, { signal }),
    enabled: Boolean(stationId),
    staleTime: 30_000,
  });
}
```

Problema original	Solución aplicada
Dependencias vacías / <code>stationId</code> desactualizado	<code>stationId</code> forma parte de la <code>queryKey</code> : un cambio de estación es, para React Query, una consulta distinta y dispara un fetch nuevo automáticamente.
Fetch duplicado	React Query dedupea peticiones concurrentes con la misma <code>queryKey</code> y respeta <code>staleTime</code> para no repetir peticiones innecesarias.
Condiciones de carrera	Al iniciar una nueva consulta, React Query cancela la anterior mediante el <code>AbortSignal</code> inyectado en <code>queryFn</code> ; el estado final siempre corresponde a la última selección.
Fetch mientras no hay selección	<code>enabled: Boolean(stationId)</code> evita disparar la consulta cuando aún no hay estación seleccionada.

Como alternativa sin librerías externas, se documentó también la solución equivalente con `useEffect` y `AbortController` puro: dependencias correctas (`[stationId]`), una función de limpieza que cancela la petición en curso, y manejo explícito de errores que ignora `AbortError`. El detalle completo de ambas soluciones, con ejemplos de código, se encuentra en `docs/BUGFIX.md` dentro del repositorio.

8. Caso situacional: trabajo en equipo

Caso planteado: el equipo de diseño cambia el layout de tarjetas a tabla, y el CMS renombra el campo `status` a `state`.

8.1 ¿Cómo minimizar el impacto del cambio?

El impacto se minimiza aislando cada cambio en la capa que le corresponde. El cambio visual (tarjetas a tabla) se resuelve dentro de StationList, el único componente que decide cómo se renderiza el listado; ningún otro módulo se entera del cambio. El cambio de contrato (status a state) se resuelve en el adaptador de services/api.ts, el único punto que conoce el nombre exacto que expone el backend; el resto de la aplicación continúa usando Station.status tal como está definido en el contrato interno (types/index.ts). En otras palabras: la interfaz depende del contrato interno, nunca directamente del contrato externo del CMS.

8.2 ¿Cómo coordinar el contrato con backend / CMS?

- **Types como contrato compartido** — types/index.ts funciona como fuente de verdad del frontend; idealmente generado desde un esquema compartido (OpenAPI / JSON Schema) para evitar divergencias silenciosas.
- **Mappers explícitos** — cada respuesta cruda del backend pasa por una función de mapeo antes de llegar a cualquier hook o componente, de modo que un cambio de nombre de campo se resuelve modificando una sola función.
- **Versionado del API** — los cambios disruptivos se exponen bajo una ruta o header versionado (por ejemplo /v2/stations) en vez de mutar el contrato vigente.
- **Comunicación** — un cambio de contrato se trata como un cambio de interfaz pública: se documenta, se anuncia con antelación y, para campos centrales, se sostiene un periodo de compatibilidad doble antes de retirar el campo anterior.

8.3 ¿Qué se haría para que estos cambios no rompan producción?

- **Tests de contrato** — un test unitario del mapper que falle explícitamente si el CMS deja de enviar el campo esperado.
- **Tipado estricto** — con strict: true, un cambio de nombre en types/index.ts marca en rojo cada archivo que aún use el nombre anterior, antes de llegar a runtime.
- **Despliegue progresivo** — el nuevo layout se libera detrás de un flag, probado con un subconjunto de usuarios antes de reemplazar el diseño de tarjetas para todos.
- **Cobertura de tests como red de seguridad** — los tests existentes sobre selección de estación y cambio de estado detectan de inmediato si la migración a tabla rompe la interacción.
- **Monitoreo post-despliegue** — registro de errores de parseo de la respuesta del CMS para detectar en minutos, no en reportes de usuario, un cambio de forma inesperado.
- **Compatibilidad temporal** — el mapper acepta ambos nombres de campo durante la ventana de migración, evitando downtime por despliegues desincronizados entre frontend y backend.

El desarrollo completo de estas respuestas se encuentra en docs/DECISIONES_TECNICAS.md dentro del repositorio.

9. Estrategia de calidad

9.1 Testing

La suite de pruebas está compuesta por 27 tests distribuidos en cuatro niveles, con una cobertura superior al 90% en la mayoría de los módulos:

Nivel	Alcance
Componente	StationCard — render, selección, toggle de estado y deshabilitado durante actualización.
Hook con mock de API	useStationServices — habilitación condicional, éxito, error y refetch al cambiar de estación.
Integración de página	StationsPage — dashboard, búsqueda, deep-linking, flujo de confirmación al inactivar (incluyendo cancelar) y notificaciones toast.
Accesibilidad automatizada	jest-axe sobre la página principal, el modal de confirmación y las tarjetas de estación — cero violaciones WCAG detectadas.
Servicio / utilitario	Join estación-servicio, persistencia del cambio de estado, cancelación con AbortController y exportación a CSV.

9.2 Accesibilidad

- Íconos de servicio con aria-label y title — la información no depende únicamente del color o la forma.
- Toggle de estado implementado con role="switch" y aria-checked.
- Estados de carga anunciados con role="status" y errores con role="alert".
- Navegación e interacción completas por teclado, con foco visible en todos los controles interactivos.
- Verificación automatizada con jest-axe integrada a la suite de tests, ejecutada en cada corrida.

9.3 CI/CD y automatización

El repositorio incorpora un pipeline de integración continua (.github/workflows/ci.yml) que se ejecuta en cada push y pull request contra la rama principal:

1. Instalación reproducible de dependencias (npm ci).
2. Lint del código (ESLint + typescript-eslint, cero advertencias permitidas).
3. Ejecución de la suite de tests con reporte de cobertura.
4. Publicación del reporte de cobertura como artefacto descargable.
5. Build de producción.

De forma complementaria, Husky y lint-staged ejecutan ESLint localmente sobre los archivos modificados antes de completar cada commit, y Dependabot revisa semanalmente las dependencias del proyecto en busca de actualizaciones y vulnerabilidades conocidas.

10. Manual de instalación

Requisitos previos: Node.js 18 o superior y npm 9 o superior.

```
# 1. Clonar el repositorio
git clone <URL_DEL_REPOSITORIO>
cd contenido-por-estacion

# 2. Instalar dependencias
npm install

# 3. Levantar el entorno de desarrollo
npm run dev
# -> abre http://localhost:5173

# 4. Ejecutar la suite de tests
npm run test

# 5. Ejecutar tests con reporte de cobertura
npm run test:coverage

# 6. Compilar para producción
npm run build

# 7. Previsualizar el build de producción
npm run preview

# 8. Verificar calidad de código (lint)
npm run lint
```

11. Manual de usuario

1. Al ingresar a la aplicación se muestra el listado de estaciones con su nombre, código y estado (activa / inactiva), junto a un panel de resumen con el total de estaciones, activas e inactivas.
2. Es posible filtrar el listado escribiendo en el buscador (por nombre o código) y ordenarlo por nombre o por estado; el atajo de teclado “/” enfoca el buscador desde cualquier parte de la pantalla.
3. Al hacer clic sobre el nombre de una estación, el panel “Servicios” carga los íconos de los servicios que ofrece esa estación. Cada ícono conserva el nombre del servicio como texto accesible para lectores de pantalla.
4. El interruptor a la derecha de cada estación permite activar o inactivar la estación sin salir del listado. Inactivar una estación solicita confirmación en un cuadro de diálogo, dado que es la acción de mayor impacto operativo; reactivar no la requiere. El cambio se refleja de inmediato y se revierte automáticamente si la operación falla, mostrando una notificación.
5. El botón “Exportar CSV” descarga el listado tal como se está visualizando (respetando el filtro y el orden aplicados).
6. El interruptor de tema en la esquina superior derecha alterna entre modo claro y oscuro; la preferencia se recuerda entre visitas.

7. Toda la interacción es operable por teclado (Tab y Enter / Espacio) y cada control interactivo cuenta con foco visible.

12. Conclusiones

La solución entregada cumple la totalidad de los requisitos funcionales y no funcionales descritos en el enunciado de la prueba técnica: listado y selección de estaciones, representación de servicios mediante ícono, modificación del estado operativo, arquitectura por capas, fetching con React Query, cobertura de pruebas y accesibilidad mínima. El bug de useEffect se analizó, corrigió y documentó con dos soluciones alternativas, y el caso situacional de coordinación con backend/CMS se resolvió con foco en minimizar el acoplamiento entre capas.

Adicionalmente, la solución incorpora un conjunto de prácticas que acercan el entregable al estándar de un producto listo para producción: manejo explícito de errores (Error Boundary), accesibilidad verificada de forma automatizada (jest-axe), integración continua con publicación de cobertura, pre-commit hooks, y una arquitectura pensada explícitamente para absorber cambios de contrato sin propagar el impacto al resto de la aplicación — el mismo principio descrito en la respuesta al caso situacional se aplicó de forma consistente en todo el desarrollo.

Anexos

Anexo A. Documentación complementaria en el repositorio

- **README.md** — manual de instalación, manual de usuario y descripción de arquitectura.
- **docs/DECISIONES_TECNICAS.md** — desarrollo completo de las respuestas al caso situacional.
- **docs/BUGFIX.md** — análisis y solución detallada del bug de useEffect, incluida la alternativa con AbortController.
- **CHANGELOG.md** — historial de cambios del proyecto en formato Keep a Changelog.
- **public/brand/README.md** — instrucciones para actualizar el logotipo de marca en el header.

Anexo B. Scripts disponibles

Comando	Descripción
npm run dev	Levanta el servidor de desarrollo
npm run build	Compila la aplicación para producción
npm run preview	Sirve localmente el build de producción
npm run test	Ejecuta la suite de tests una vez
npm run test:watch	Ejecuta los tests en modo observador
npm run test:coverage	Ejecuta los tests y genera el reporte de cobertura
npm run lint	Verifica el código con ESLint

Anexo C. Datos de contacto del proyecto

Campo	Valor
Proyecto	AppTerpel
Área	Dirección Canales Digitales
Repositorio	GitHub — Prueba-Tecnica-Ingeniero-de-apliaciones-de-Negocio